



Operations Research

Publication details, including instructions for authors and subscription information:
<http://pubsonline.informs.org>

Mixed-Integer Optimization with Constraint Learning

Donato Maragno; , Holly Wiberg; , Dimitris Bertsimas, Ş. İlker Birbil; , Dick den Hertog; , Adejuyigbe O. Fajemisin;

To cite this article:

Donato Maragno; , Holly Wiberg; , Dimitris Bertsimas, Ş. İlker Birbil; , Dick den Hertog; , Adejuyigbe O. Fajemisin; (2025) Mixed-Integer Optimization with Constraint Learning. Operations Research 73(2):1011-1028. <https://doi.org/10.1287/opre.2021.0707>

Full terms and conditions of use: <https://pubsonline.informs.org/Publications/Librarians-Portal/PubsOnLine-Terms-and-Conditions>

This article may be used only for the purposes of research, teaching, and/or private study. Commercial use or systematic downloading (by robots or other automatic processes) is prohibited without explicit Publisher approval, unless otherwise noted. For more information, contact permissions@informs.org.

The Publisher does not warrant or guarantee the article's accuracy, completeness, merchantability, fitness for a particular purpose, or non-infringement. Descriptions of, or references to, products or publications, or inclusion of an advertisement in this article, neither constitutes nor implies a guarantee, endorsement, or support of claims made of that product, publication, or service.

Copyright © 2023, INFORMS

Please scroll down for article—it is on subsequent pages



With 12,500 members from nearly 90 countries, INFORMS is the largest international association of operations research (O.R.) and analytics professionals and students. INFORMS provides unique networking and learning opportunities for individual professionals, and organizations of all types and sizes, to better understand and use O.R. and analytics tools and methods to transform strategic visions and achieve better outcomes. For more information on INFORMS, its publications, membership, or meetings visit <http://www.informs.org>

Methods

Mixed-Integer Optimization with Constraint Learning

Donato Maragno,^a Holly Wiberg,^b Dimitris Bertsimas,^{c,*} Ş. Ilker Birbil,^a Dick den Hertog,^a Adejuyigbe O. Fajemisin^a

^a Amsterdam Business School, University of Amsterdam, 1018 TV Amsterdam, Netherlands; ^b Heinz College of Information Systems and Public Policy, Carnegie Mellon University, Pittsburgh, Pennsylvania 15213; ^c Sloan School of Management, Massachusetts Institute of Technology, Cambridge, Massachusetts 02139

*Corresponding author

Contact: d.maragno@uva.nl,  <https://orcid.org/0000-0001-5857-8807> (DM); hwiberg@andrew.cmu.edu,  <https://orcid.org/0000-0002-4251-905X> (HW); dbertsim@mit.edu,  <https://orcid.org/0000-0002-1985-1003> (DB); s.i.birbil@uva.nl,  <https://orcid.org/0000-0001-7472-7032> (ŞİB); d.denhertog@uva.nl,  <https://orcid.org/0000-0002-1829-855X> (DdH); a.o.fajemisin2@uva.nl,  <https://orcid.org/0000-0003-0648-3800> (AOF)

Received: November 4, 2021

Revised: January 18, 2023; September 24, 2023

Accepted: October 20, 2023

Published Online in Articles in Advance: December 1, 2023

Area of Review: Optimization

<https://doi.org/10.1287/opre.2021.0707>

Copyright: © 2023 INFORMS

Abstract. We establish a broad methodological foundation for mixed-integer optimization with learned constraints. We propose an end-to-end pipeline for data-driven decision making in which constraints and objectives are directly learned from data using machine learning, and the trained models are embedded in an optimization formulation. We exploit the mixed-integer optimization representability of many machine learning methods, including linear models, decision trees, ensembles, and multilayer perceptrons, which allows us to capture various underlying relationships between decisions, contextual variables, and outcomes. We also introduce two approaches for handling the inherent uncertainty of learning from data. First, we characterize a decision trust region using the convex hull of the observations to ensure credible recommendations and avoid extrapolation. We efficiently incorporate this representation using column generation and propose a more flexible formulation to deal with low-density regions and high-dimensional data sets. Then, we propose an ensemble learning approach that enforces constraint satisfaction over multiple bootstrapped estimators or multiple algorithms. In combination with domain-driven components, the embedded models and trust region define a mixed-integer optimization problem for prescription generation. We implement this framework as a Python package (OptiCL) for practitioners. We demonstrate the method in both World Food Programme planning and chemotherapy optimization. The case studies illustrate the framework's ability to generate high-quality prescriptions and the value added by the trust region, the use of ensembles to control model robustness, the consideration of multiple machine learning methods, and the inclusion of multiple learned constraints.

Funding: This work was supported by the Dutch Scientific Council [Grant OCENW.GROOT.2019.015] and the National Science Foundation [Grant 174530]. Additionally, H. Wiberg was supported by the National Science Foundation Graduate Research Fellowship [Grant 174530].

Supplemental Material: The online appendix is available at <https://doi.org/10.1287/opre.2021.0707>.

Keywords: mixed-integer optimization • machine learning • constraint learning • prescriptive analytics

1. Introduction

Mixed-integer optimization (MIO) is a powerful tool that allows us to optimize a given objective subject to various constraints. This general problem statement of optimizing under constraints is nearly universal in decision-making settings. Some problems have readily quantifiable and explicit objectives and constraints, in which case MIO can be directly applied. The situation becomes more complicated, however, when the constraints and/or objectives are not explicitly known.

For example, suppose we deal with cancerous tumors and want to prescribe a treatment regimen with a limit on toxicity; we may have observational data on treatments

and their toxicity outcomes, but we have no natural function that relates the treatment decision to its resultant toxicity. We may also encounter constraints that are not directly quantifiable. Consider a setting where we want to recommend a diet, defined by a combination of foods and quantities, that is sufficiently “palatable.” Palatability cannot be written as a function of the food choices, but we may have qualitative data on how well people “like” various potential dietary prescriptions. In both these examples, we cannot directly represent the outcomes of interest as functions of our decisions, but we have *data* that relate the outcomes and decisions. This raises a question: How can we consider data to learn these functions?

In this work, we introduce Optimization with Constraint Learning (Opt iCL) to tackle the challenge of data-driven decision making through a combined machine learning (ML) and MIO approach. ML allows us to learn functions that relate decisions to outcomes of interest directly through data. Importantly, many popular ML methods result in functions that are MIO representable, meaning that they can be embedded into MIO formulations. This MIO-representable class includes both linear and non-linear models, allowing us to capture a broad set of underlying relationships in the data. Although the idea of learning functions directly from data are core to the field of ML, data are often underused in MIO settings due to the need for functional relationships between decision variables and outcomes. We seek to bridge this gap through *constraint learning*; we propose a general framework that allows us to learn constraints and objectives directly from data, using ML, and to optimize decisions accordingly, using MIO. Once the learned constraints have been incorporated into the larger MIO, we can solve the problem directly using off-the-shelf solvers.

The term *constraint learning*, used several times throughout this work, captures both constraints and objective functions. We are fundamentally learning functions to relate our decision variables to the outcome(s) of interest. The predicted values can then either be incorporated as constraints or objective terms; the model learning and embedding procedures remain largely the same. For this reason, we refer to them both under the same umbrella of *constraint learning*. We describe this further in Section 2.2.

1.1. Literature Review

Previous work has demonstrated the use of various ML methods in MIO problems and their utility in different application domains. The simplest of these methods is the regression function, as the approach is easy to understand and easy to implement. Given a regression function learned from data, the process of incorporating it into an MIO model is straightforward, and the final model does not require complex reformulations. As an example, Bertsimas et al. (2016) use regression models and MIO to develop new chemotherapy regimens based on existing data from previous clinical trials. Kleijnen (2018) provides further information on this subject.

More complex ML models have also been shown to be MIO representable, although more effort is required to represent them than simple regression models. Neural networks that use the rectified linear unit (ReLU) activation function can be represented using binary variables and big-M formulations (Amos et al. 2017, Grimstad and Andersson 2019, Anderson et al. 2020, Chen et al. 2020, Spyros 2020, Murzakhanov et al. 2022). Where other activation functions are used (Gutierrez-Martinez et al. 2011, Lombardi et al. 2017, Schweidtmann and Mitsos 2019), the MIO representation of neural

networks is still possible, provided the solvers are capable of handling these functions.

With decision trees, each path in the tree from root to leaf node can be represented using one or more constraints (Bonfietti et al. 2015, Verwer et al. 2017, Halilbasic et al. 2018). The number of constraints required to represent decision trees is a function of the tree size, with larger trees requiring more linearizations and binary variables. The advantage here, however, is that decision trees are known to be highly interpretable, which is often a requirement of ML in critical application settings (Thams et al. 2017). Random forests (Mišić 2020, Biggs et al. 2021) and other tree ensembles (Cremer et al. 2019) have also been used in MIO in the same way as decision trees, with one set of constraints for each tree in the forest/ensemble along with one or more additional aggregate constraints.

Data for constraint learning can either contain information on continuous data, feasible and infeasible states (two-class data), or only one state (one-class data). The problem of learning functions from one-class data and embedding them into optimization models has been recently investigated with the use of decision trees (Kudła and Pawlak 2018), genetic programming (Pawlak and Krawiec 2019), local search (Sroka and Pawlak 2018), evolutionary strategies (Pawlak 2019), and a combination of clustering, principal component analysis and wrapping ellipsoids (Pawlak and Litwiniuk 2021).

The previously selected applications generally involve a single function to be learned and a fixed ML method for the model choice. Verwer et al. (2017) use two model classes (decision trees and linear models) in a specific auction design application, but in this case, the models were determined a priori. Some authors have presented a more general framework of embedding learned ML models in optimization problems such as JANOS (Bergman et al. 2022) and Empirical Model Learning (EML) (Lombardi et al. 2017), but in practice these works are restricted to limited problem structures and learned model classes. We take a broader perspective, proposing a comprehensive end-to-end pipeline that encompasses the full ML and optimization components of a data-driven decision making problem. In contrast to EML and JANOS, Opt iCL supports a wider variety of predictive models—neural networks (with ReLU), linear regression, logistic regression, decision trees, random forests, gradient boosted trees, and linear support vector machines. Opt iCL is also more flexible than JANOS, as it can handle predictive models as constraints, and it also incorporates new concepts to deal with uncertainty in the ML models. A comparison of Opt iCL against JANOS and EML on two test problems is shown in Online Appendix E.

Our work falls under the umbrella of prescriptive analytics. Bertsimas and Kallus (2020) and Elmachtoub and Grigas (2021) leverage ML model predictions as inputs into an optimization problem. Our approach is

distinct from existing work in that we directly embed ML models rather than extracting predictions, allowing us to optimize our decisions over the model. In the broadest sense, our framework relates to work that jointly harnesses ML and MIO, an area that has garnered significant interest in recent years in both the optimization and machine learning communities (Bengio et al. 2021).

1.2. Contributions

Our work unifies several research areas in a comprehensive manner. Our key contributions are as follows:

1. We develop an end-to-end framework that takes data and directly implements model training, model selection, integration into a larger MIO, and ultimately optimization. We make this available as an open-source software, OptiCL (Optimization with Constraint Learning) to provide a practitioner-friendly tool for making better data-driven decisions. The code is available at <https://github.com/hwiberg/OptiCL>. The software encompasses the full ML and optimization pipeline with the goal of being accessible to end users and extensible by technical researchers. Our framework natively supports models for both regression and classification functions and handles constraint learning in cases with both one-class and two-class data. We implement a cross-validation procedure for function learning that selects from a broad set of model classes. We also implement the optimization procedure in the generic mathematical modeling library Pyomo, which supports various state-of-the-art solvers. We introduce two approaches for handling the inherent uncertainty when learning from data. First, we propose an ensemble learning approach that enforces constraint satisfaction over an ensemble of multiple bootstrapped estimators or multiple algorithms, yielding more robust solutions. This addresses a shortcoming of existing approaches to embedding trained ML models, which rely on a single point prediction: in the case of learned constraints, model misspecification can lead to infeasibility. Additionally, we restrict solutions to lie within a trust region, defined as the domain of the training data, which leads to better performance of the learned constraints. We offer several improvements to a basic convex hull formulation, including a clustering heuristic and a column selection algorithm that significantly reduce computation time. We also propose an enlargement of the convex hull that allows for exploration of solutions outside of the observed bounds. Both the ensemble model wrapper and trust region enlargement are controlled by parameters that allow an end user to directly tradeoff the conservativeness of the constraint satisfaction.

2. We demonstrate the power of our method in two real-world case studies, using data from the World Food Programme and chemotherapy clinical trials. We

pose relevant questions in the respective areas and formalize them as constraint learning problems. We implement our framework and subsequently evaluate the quantitative performance and scalability of our methods in these settings.

2. Embedding Predictive Models

Suppose we have data $\mathcal{D} = \{(\bar{x}_i, \bar{w}_i, \bar{y}_i)\}_{i=1}^N$, with observed treatment decisions $\bar{x}_i$, contextual information $\bar{w}_i$, and outcomes of interest $\bar{y}_i$ for sample i . Following the guidelines proposed in Fajemisin et al. (2023), we present a framework that, given data $\mathcal{D}$, learns functions for the outcomes of interest (y) that are to be constrained or optimized. These learned representations can then be used to generate predictions for a new observation with context w . Figure 1 outlines the complete pipeline, which is detailed in the following sections.

2.1. Conceptual Model

Given the decision variable $x \in \mathbb{R}^n$ and the fixed feature vector $w \in \mathbb{R}^p$, we propose model $M(w)$

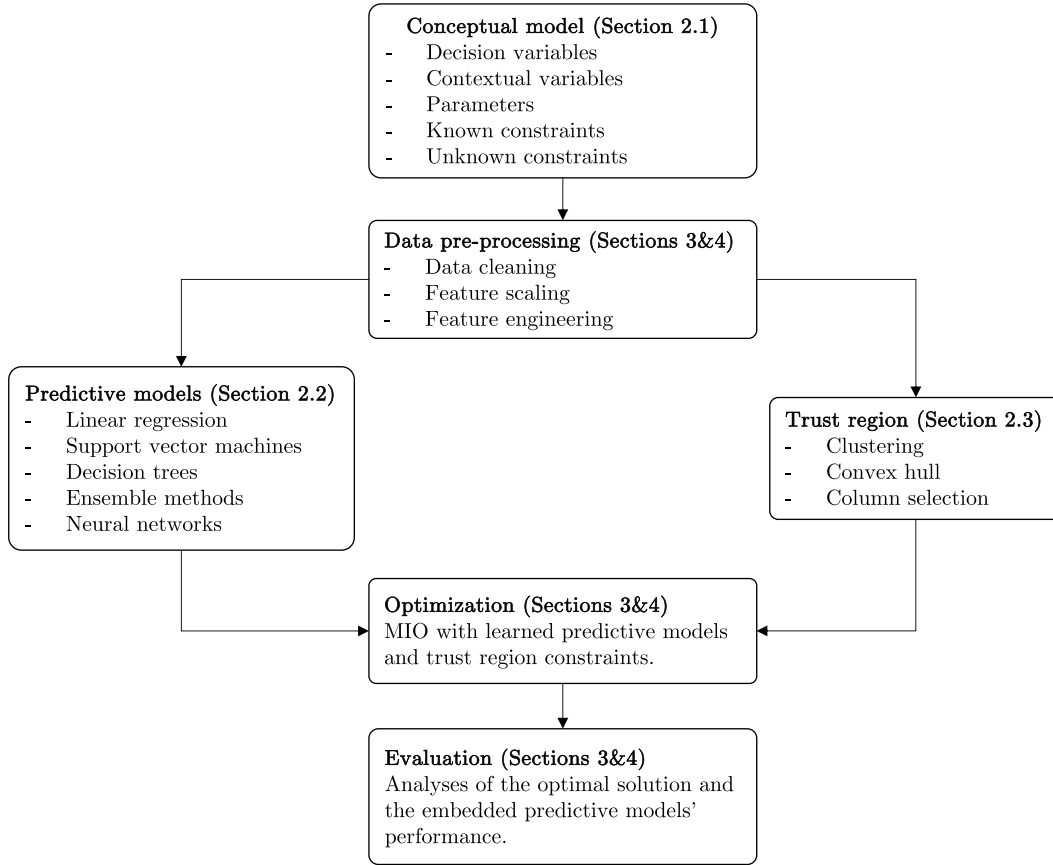
$$\begin{aligned} \min_{x \in \mathbb{R}^n, y \in \mathbb{R}^k} \quad & f(x, w, y) \\ \text{s.t.} \quad & g(x, w, y) \leq 0, \\ & y = \hat{h}_{\mathcal{D}}(x, w), \\ & x \in \mathcal{X}(w), \end{aligned} \quad (1)$$

where $f(\cdot, w, \cdot) : \mathbb{R}^{n+k} \mapsto \mathbb{R}$, $g(\cdot, w, \cdot) : \mathbb{R}^{n+k} \mapsto \mathbb{R}^m$, and $\hat{h}_{\mathcal{D}}(\cdot, w) : \mathbb{R}^n \mapsto \mathbb{R}^k$. Explicit forms of f and g are known but they may still depend on the predicted outcome y . Here, $\hat{h}_{\mathcal{D}}(x, w)$ represents the predictive models, one per outcome of interest, which are ML models trained on $\mathcal{D}$. Although our subsequent discussion mainly revolves around linear functions, we acknowledge the significant progress in nonlinear (convex) integer solvers. Our discussion can be easily extended to nonlinear models that can be tackled by those ever-improving solvers.

The embedding of a single learned outcome may require multiple constraints and auxiliary variables; the embedding formulations are described in Section 2.2. For simplicity, we omit $\mathcal{D}$ in further notation of $\hat{h}$ but note that all references to $\hat{h}$ implicitly depend on the data used to train the model. Finally, the set $\mathcal{X}(w)$ defines the trust region, that is, the set of solutions for which we trust the embedded predictive models. In Section 2.3, we provide a detailed description of how the trust region $\mathcal{X}(w)$ is obtained from the observed data. We refer to the final MIO formulation with the embedded constraints and variables as EM(w).

Model $M(w)$ is quite general and encompasses several important *constraint learning* classes:

1. **Regression.** When the trained model results from a regression problem, it can be constrained by a specified upper bound τ , that is, $g(y) = y - \tau \leq 0$, or lower

Figure 1. Constraint Learning and Optimization Pipeline

bound τ , that is, $g(y) = -y + \tau \leq 0$. If y is a vector (i.e., multioutput regression), we can likewise provide a threshold vector τ for the constraints.

2. Classification. If the trained model is obtained with a binary classification algorithm, in which the data are labeled as “feasible” (1) or “infeasible” (0), then the prediction is generally a probability $y \in [0, 1]$. We can enforce a lower bound on the feasibility probability, that is, $y \geq \tau$. A natural choice of τ is 0.5, which can be interpreted as enforcing that the result is more likely feasible than not. This can also extend to the multiclass setting, say k classes, in which the output y is a k -dimensional unit vector, and we apply the constraint $y_i \geq \tau$ for whichever class i is desired. When multiple classes are considered to be feasible, we can add binary variables to ensure that a solution is feasible, only if it falls in one of these classes with sufficiently high probability.

3. Objective function. If the objective function has a term that is also learned by training an ML model, then we can introduce an auxiliary variable $t \in \mathbb{R}$, and add it to the objective function along with an epigraph constraint. Suppose for simplicity that the model involves a single learned objective function, $\hat{h}$, and no learned

constraints. Then the general model becomes

$$\begin{aligned}
 \min_{x \in \mathbb{R}^n, y \in \mathbb{R}, t \in \mathbb{R}} \quad & t \\
 \text{s.t.} \quad & g(x, w) \leq 0, \\
 & y = \hat{h}(x, w), \\
 & y - t \leq 0, \\
 & x \in \mathcal{X}(w).
 \end{aligned}$$

Although we have rewritten the problem to show the generality of our model, it is quite common in practice to use y in the objective and omit the auxiliary variable t .

We observe that constraints on learned outcomes can be applied in two ways depending on the model training approach. Suppose that we have a continuous scalar outcome y to learn, and we want to impose an upper bound of $\tau \in \mathbb{R}$ (it may also be a lower bound without loss of generality). The first approach is called *function learning* and concerns all cases where we learn a regression function $\hat{h}(x, w)$ without considering the feasibility threshold (τ). The resultant model returns a predicted value $y \in \mathbb{R}$. The threshold is then applied as a constraint in the optimization model as $y \leq \tau$. Alternatively, we could use the feasibility threshold τ to binarize the outcome of each sample in $\mathcal{D}$ into feasible and infeasible,

that is $\bar{y}_i := \mathbb{I}(\bar{y}_i \leq \tau)$, $i = 1, \dots, N$, where $\mathbb{I}$ stands for the indicator function. After this relabeling, we train a binary classification model $\hat{h}(x, w)$ that returns a probability $y \in [0, 1]$. This approach, called *indicator function learning*, does not require any further use of the feasibility threshold τ in the optimization model because the predictive models directly encode feasibility.

The function learning approach is particularly useful when we are interested in varying the threshold τ as a model parameter. Additionally, if the fitting process is expensive and therefore difficult to perform multiple times, learning an indicator function for each potential τ might be infeasible. In contrast, the indicator function learning approach is necessary when the raw data contains binary labels rather than continuous outcomes, and thus we have no ability to select or vary τ .

2.2. MIO-Representable Predictive Models

Our framework is enabled by the ability to embed learned predictive models into an MIO formulation with linear constraints. This is possible for many classes of ML models, ranging from linear models to ensembles, and from support vector machines to neural networks. In this section, we outline the embedding procedure for decision trees, tree ensembles, and neural networks to illustrate the approach. We include additional technical details and formulations for these methods, along with linear regression and support vector machines, in Online Appendix A.

In all cases, the model has been *pretrained*; we embed the trained model $\hat{h}(x, w)$ into our larger MIO formulation to allow us to constrain or optimize the resultant predicted value. Consequently, the optimization model is not dependent on the complexity of the model training procedure, but solely the size of the final trained model. Without loss of generality, we assume that y is one-dimensional; that is, we are learning a single model, and this model returns a scalar and not a multioutput vector.

All the methods here can be used to learn constraints that apply upper or lower bounds to y or to learn y that we incorporate as part of the objective. We present the model embedding procedure for both cases when $\hat{h}(x, w)$ is a continuous or a binary predictive model, where relevant. We assume that either regression or classification models can be used to learn feasibility constraints, as described in Section 2.1.

2.2.1. Decision Trees. Decision trees partition observations into distinct *leaves* through a series of *feature splits*. These algorithms are popular in predictive tasks due to their natural interpretability and ability to capture non-linear interactions among variables. Breiman et al. (1984) first introduced classification and regression trees (CART), which constructs trees through parallel splits in the feature space. Decision tree algorithms have

subsequently been adapted and extended. Bertsimas and Dunn (2017) propose an alternative decision tree algorithm, optimal classification trees (and optimal regression trees), that improves on the basic decision tree formulation through an optimization framework that approximates globally optimal trees. Optimal trees also support multifeature splits, referred to as *hyper-plane splits*, that allow for splits on a linear combination of features (Bertsimas and Dunn 2018).

A generic decision tree of depth 2 is shown in Figure 2. A split at node i is described by an inequality $A_i^\top x \leq b_i$. We assume that A can have multiple nonzero elements, in which we have the hyper-plane split setting; if there is only one nonzero element, this creates a parallel (single feature) split. Each terminal node j (i.e., leaf) yields a prediction (p_j) for its observations. In the case of regression, the prediction is the average value of the training observations in the leaf, and in binary classification, the prediction is the proportion of leaf members with the feasible class. Each leaf can be described as a polyhedron, namely a set of linear constraints that must be satisfied by all leaf members. For example, for node 3, we define $\mathcal{P}_3 = \{x : A_1^\top x \leq b_1, A_2^\top x \leq b_2\}$.

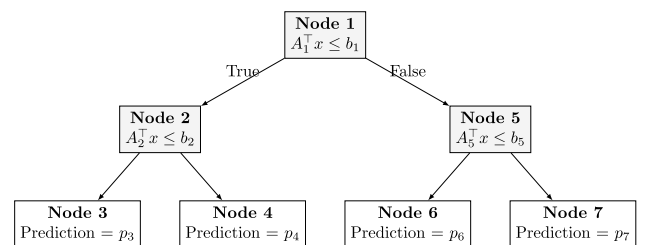
Suppose that we wish to constrain the predicted value of this tree to be at most τ , a fixed constant. After obtaining the tree in Figure 2, we can identify which paths satisfy the desired bound ($p_i \leq \tau$). Suppose that p_3 and p_6 do satisfy the bound, but p_4 and p_7 do not. In this case, we can enforce that our solution belongs to $\mathcal{P}_3$ or $\mathcal{P}_6$. This same approach applies if we only have access to two-class data (feasible vs. infeasible); we can directly train a binary classification algorithm and enforce that the solution lies within one of the “feasible” prediction leaves (determined by a set probability threshold).

If the decision tree provides our only learned constraint, we can decompose the problem into multiple separate MIOs, one per feasible leaf. The conceptual model for the subproblem of leaf i then becomes

$$\begin{aligned} \min_x \quad & f(x, w) \\ \text{s.t.} \quad & g(x, w) \leq 0, \\ & (x, w) \in \mathcal{P}_i, \end{aligned}$$

where the learned constraints for leaf i 's subproblem are implicitly represented by the polyhedron $\mathcal{P}_i$. These

Figure 2. Decision Tree of Depth 2 with Four Terminal Nodes (Leaves)



subproblems can be solved in parallel, and the minimum across all subproblems is obtained as the optimal solution. Furthermore, if all decision variables x are continuous, these subproblems are linear optimization problems (LOs), which can provide substantial computational gains. This is explored further in Online Appendix A.2.

In the more general setting where the decision tree forms one of many constraints, or we are interested in varying the τ limit within the model, we can directly embed the model into a larger MIO. We add binary variables representing each leaf, and set y to the predicted value of the assigned leaf. An observation can only be assigned to a leaf, if it obeys all of its constraints; the structure of the tree guarantees that exactly one path will be fully satisfied, and thus the leaf assignment is uniquely determined. A solution belonging to $\mathcal{P}_3$ will inherit $y = p_3$. Then, y can be used in a constraint or objective. The full formulation for the embedded decision tree is included in Online Appendix A.2. This formulation is similar to the proposal in Verwer et al. (2017). Both approaches have their own merits: although the Verwer formulation includes fewer constraints in the general case, our formulation is more efficient in the case where the problem can be decomposed into individual subproblems (as described previously).

2.2.2. Ensemble Methods. Ensemble methods, such as random forests (RF) and gradient-boosting machines (GBM) consist of many decision trees that are aggregated to obtain a single prediction for a given observation. These models can thus be implemented by embedding many “submodels” (Breiman 2001). Suppose we have a forest with P trees. Each tree can be embedded as a single decision tree (see previous paragraph) with the constraints from Online Appendix A.2, which yields a predicted value y_i .

RF models typically generate predictions by taking the average of the predictions from the individual trees:

$$y = \frac{1}{P} \sum_{i=1}^P y_i.$$

This can then be used as a term in the objective or constrained by an upper bound as $y \leq \tau$; this can be done equivalently for a lower bound. In the classification setting, the prediction averages the probabilities returned by each model ($y_i \in [0, 1]$), which can likewise be constrained or optimized.

Alternatively, we can further leverage the fact that unlike the other model classes, which return a single prediction, the RF model generates P predictions, one per tree. We can impose a violation limit across the individual P estimators as proposed in Section 3.1.

In the case of GBM, we have an ensemble of base-learners which are not necessarily decision trees. The

model output is then computed as

$$y = \sum_{i=1}^P \beta_i y_i,$$

where y_i is the predicted value of the i th regression model $\hat{h}_i(x, w)$, and β_i is the weight associated with the prediction. Although trees are typically used as base learners, in theory, we might use any of the MIO-representable predictive models discussed in this section.

2.2.3. Neural Networks. We implement multilayer perceptrons (MLP) with a rectified linear unit (ReLU) activation function, which form an MIO-representable class of neural networks (Grimstad and Andersson 2019, Anderson et al. 2020). These networks consist of an input layer, $L - 2$ hidden layer(s), and an output layer. This nonlinear transformation of the input space over multiple nodes (and layers) using the ReLU operator ($v = \max\{0, x\}$) allows MLPs to capture complex functions that other algorithms cannot adequately encode, making them a powerful class of models.

Critically, the ReLU operator, $v = \max\{0, x\}$, can be encoded using linear constraints, as detailed in Online Appendix A.3. The constraints for an MLP network can be generated recursively starting from the input layer, which allows us to embed a trained MLP with an arbitrary number of hidden layers and nodes into an MIO. We refer to Online Appendix A.3 for details on the embedding of regression, binary classification, and multiclass classification MLP variants.

2.3. Convex Hull as Trust Region

As the optimal solutions of optimization problems are often at the extremes of the feasible region, this can be problematic for the validity of the trained ML model. Generally speaking, the accuracy of a predictive model deteriorates for points that are further away from the data points in $\mathcal{D}$ (Goodfellow et al. 2015). To mitigate this problem, we elaborate on the idea proposed by Biggs et al. (2021) to use the convex hull (CH) of the data set as a trust region to prevent the predictive model from extrapolating. According to Ebert et al. (2014), when data are enclosed by a boundary of convex shape, the region inside this boundary is known as an interpolation region. This interpolation region is also referred to as the CH, and by excluding solutions outside the CH, we prevent extrapolation. If $X = \{\hat{x}_i\}_{i=1}^N$ is the set of observed input data with $\hat{x}_i = (\bar{x}_i, \bar{w}_i)$, we define the trust region as the CH of this set and denote it by $\text{CH}(X)$. Recall that $\text{CH}(X)$ is the smallest convex polytope that contains the set of points X . It is well known that computing the CH is exponential in time and space with respect to the number of samples and their dimensionality (Skiena 2008). However, because the CH is a polytope, explicit expressions for its facets are not

necessary. More precisely, $\text{CH}(X)$ is represented as

$$\text{CH}(X) = \left\{ x \mid \sum_{i \in \mathcal{I}} \lambda_i \hat{x}_i = x, \sum_{i \in \mathcal{I}} \lambda_i = 1, \lambda \geq 0 \right\}, \quad (2)$$

where $\lambda \in \mathbb{R}^N$, and $\mathcal{I} = \{1, \dots, N\}$ is the index set of samples in X .

In situations such as the one shown in Figure 3(a), $\text{CH}(X)$ includes regions with few or no data points (low-density regions). Blindly using $\text{CH}(X)$ in this case can be problematic if the solutions are found in the low-density regions. We therefore advocate the use of a two-step approach. First, clustering is used to identify distinct high-density regions, and then the trust region is represented as the union of the CHs of the individual clusters (Figure 3(b)).

We can either solve $\text{EM}(w)$ for each cluster or embed the union of the $|\mathcal{K}|$ CHs into the MIO given by

$$\bigcup_{k \in \mathcal{K}} \text{CH}(X_k) = \left\{ x \mid \sum_{i \in \mathcal{I}_k} \lambda_i \hat{x}_i = x, \sum_{i \in \mathcal{I}_k} \lambda_i = u_k, \right. \\ \left. \forall k \in \mathcal{K}, \sum_{k \in \mathcal{K}} u_k = 1, \lambda \geq 0, u \in \{0, 1\}^{|\mathcal{K}|} \right\}, \quad (3)$$

where $X_k \subseteq X$ refers to subset of samples in cluster $k \in \mathcal{K}$ with the index set $\mathcal{I}_k \subseteq \mathcal{I}$. The union of CHs requires the binary variables u_k to constrain a feasible solution to be exactly in one of the CHs. More precisely, $u_k = 1$ corresponds to the CH of the k th cluster. As we show in Section 4, solving $\text{EM}(w)$ for each cluster may be done in parallel, which has a positive impact on computation time. Both Formulations (2) and (3) assume that $\hat{x}$ is continuous. These formulations can be extended to datasets with binary, categorical and ordinal features. In the case of categorical features, extra constraints on the domain and one-hot encoding are required.

Although the CH can be represented by linear constraints, the number of variables in $\text{EM}(w)$ increases with the increase in the data set size, which may make

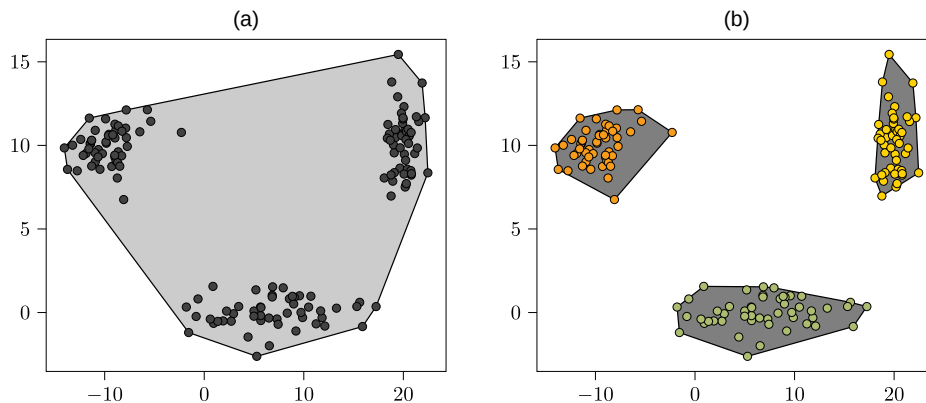
the optimization process prohibitive when the number of samples becomes too large. We therefore provide a column selection algorithm that selects a small subset of the samples. This algorithm can be directly used in the case of convex optimization problems or embedded as part of a branch-and-bound algorithm when the optimization problem involves integer variables. Figure 4 visually demonstrates the procedure; we begin with an arbitrary sample of the full data and use column selection to iteratively add samples $\hat{x}_i$ until no improvement can be found. In Online Appendix B.2, we provide a full description of the approach, as well as a formal lemma that states that in each iteration of column selection, the selected sample from X is also a vertex of $\text{CH}(X)$. In synthetic experiments, we observe that the algorithm scales well with the data set size. The computation time required by solving the optimization problem with the algorithm is near-constant and minimally affected by the number of samples in the data set. The experiments in Online Appendix B.2 show optimization with column selection to be significantly faster than a traditional approach, which makes it an ideal choice when dealing with massive data sets.

3. Uncertainty and Robustness

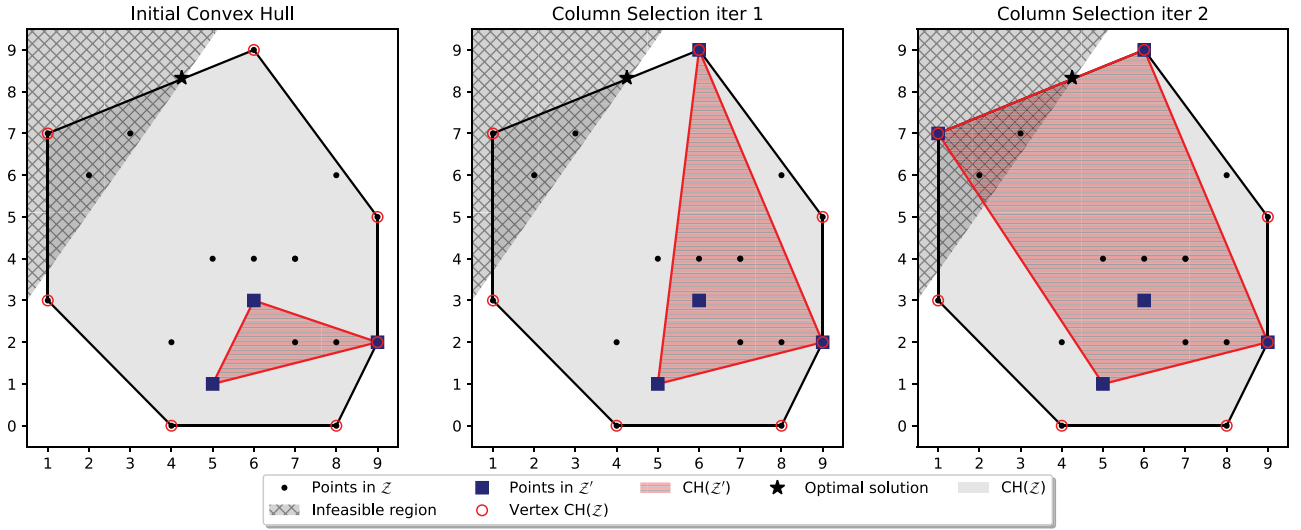
There are multiple sources of uncertainty, and consequently notions of robustness, that can be considered when embedding a trained machine learning model as a constraint. We define two types of uncertainty in Model (1).

Function Uncertainty. The first source of uncertainty is in the underlying functional form of $\hat{h}$. We do not know the ground truth relationship between (x, w) and y , and there is potential for model mis-specification. We mitigate this risk through our nonparametric model selection procedure, namely training $\hat{h}$ for a diverse set of methods (e.g., decision tree, regression, neural network) and selecting the final model using a cross-validation procedure.

Figure 3. (Color online) Use of the Two-Step Approach to Remove Low-Density Regions



Notes. (a) $\text{CH}(X)$ with single region. (b) $\text{CH}(X)$ with clustered regions.

Figure 4. (Color online) Visualization of the Column Selection Algorithm

Notes. Known and learned constraints define the infeasible region. The column selection algorithm starts using only a subset of data points (squares), $X' \subseteq X$ to define the trust region. In each iteration a vertex of $CH(X)$ is selected (hollow circle) and included in X' until the optimal solution (star) is within the feasible region, namely the convex hull of X' . Note that with column selection we do not need the complete dataset to obtain the optimal solution, but rather only a subset.

Parameter Uncertainty. Even within a single model class, there is uncertainty in the parameter estimates that define $\hat{h}$. Consider the case of linear regression. A regression estimator consists of point estimates of coefficients and an intercept term, but there is uncertainty in the estimates as they are derived from noisy data. We seek to make our model robust by characterizing this uncertainty and optimizing against it. We propose model-wrapper ensemble approaches, which are agnostic to the underlying model. The rest of this section addresses the model-wrapper approaches and a looser formulation of the trust region that prevents the optimal solution from being too conservative when the predictive models have good extrapolation performance.

3.1. Model Wrapper Approach

We begin by describing the model “wrapper” approach for characterizing uncertainty, in which we work directly with any trained models and their point predictions. Rather than obtaining our estimated outcome from a single trained predictive model, we suppose that we have P estimators. The set of estimators can be obtained by bootstrapping or by training models using entirely different methods. The uncertainty is thus characterized by different realizations of the predicted value from multiple estimators, which effectively form an ensemble.

We introduce a constraint that at most $\alpha \in [0, 1]$ proportion of the P estimators violate the constraint. Let $\hat{h}_1, \dots, \hat{h}_P$ be the individual estimators. Then $\hat{h}_i(x) \leq \tau$ in at least $1 - \alpha P$ of these estimators. This allows for a degree of robustness to individual model predictions by discarding a small number of potential outlier predictions.

Formally,

$$\frac{1}{P} \sum_{i=1}^P \mathbb{I}(y_i \leq \tau) \geq 1 - \alpha. \quad (4)$$

Note that $\alpha = 0$ enforces the bound for all estimators, yielding the most conservative estimate, whereas $\alpha = 1$ removes the constraint entirely. Constraint (4) is MIO representable:

$$y_i \leq \tau + M(1 - z_i), \quad i = 1, \dots, P$$

$$\frac{1}{P} \sum_{i=1}^P z_i \geq 1 - \alpha,$$

where $z_i \in \{0, 1\} \forall i = 1, \dots, P$, and M is a sufficiently large constant. Online Appendix A.4 includes further details on this formulation and special cases.

The violation limit concept can also be applied to estimators coming from multiple model classes, which allows us to enforce that the constraint is *generally* obeyed when modeled through distinct methods. This provides a measure of robustness to *function uncertainty*.

3.2. Enlarged Convex Hull

The use of the model wrapper approach and the trust region constraints, as defined in (2), has a direct effect on the feasible region. The better performance of the learned constraints might be balanced out by the (potentially) unnecessary conservatism of the optimal solution. Although we introduced the trust region as a set of constraints to preserve the predictive performance of the fitted constraints, Balestriero et al. (2021) show how in a high-dimensional space the generalization performance of a fitted model is typically obtained extrapolating. In

light of this evidence, we propose an ϵ -CH formulation that builds on (2) and more generally on (3). The relaxed formulation of the trust region enables the optimal solution of problem $M(w)$ to be outside $CH(X)$. Formally, we enlarge the trust region such that solutions outside $CH(X)$ are considered feasible if they fall within the hyperball, with radius ϵ , surrounding at least one of the data points in X (Figure 5 (left)). The ϵ -CH is formulated as follows:

$$\epsilon\text{-CH}(X) = \left\{ (x, s) \left| \sum_{i \in I} \lambda_i \hat{x}_i = x + s, \sum_{i \in I} \lambda_i = 1, \right. \right. \\ \left. \left. \lambda \geq 0, \|s\|_p \leq \epsilon \right\}, \quad (5)$$

with $s \in \mathbb{R}^n$, and p set equal to 1, 2, or ∞ to preserve the complexity of the optimization problem. Figure 5 (right) shows the extended region obtained with the ϵ -CH. The choice of ϵ is pivotal in the trade-off between the performance of the learned constraints and the conservatism of the optimal solution. In the next section, we demonstrate how an increase in ϵ affects both the performance of the embedded predictive models and the objective function value.

4. Case Study: A Palatable Food Basket for the World Food Programme

In this case study, we use a simplified version of the model proposed by Peters et al. (2021), which seeks to optimize humanitarian food aid. Its extended version aims to provide the World Food Programme (WFP) with a decision-making tool for long-term recovery operations, which simultaneously optimizes the food

basket to be delivered, the sourcing plan, the delivery plan, and the transfer modality of a month-long food supply. The model proposed by Peters et al. (2021) enforces that the food baskets address the nutrient gap and are palatable. To guarantee a certain level of palatability, the authors use a number of “unwritten rules” that have been defined in collaboration with nutrition experts. In this case study, we take a step further by inferring palatability constraints directly from data that reflects local people’s opinions. We use the specific case of Syria for this example. The conceptual model presents an LO structure with only the food palatability constraint to be learned. Data on palatability are generated through a simulator, but the procedure would remain unchanged if data were collected in the field, for example, through surveys. The structure of this problem, which is an LO and involves only one learned constraint, allows the following analyses: (1) the effect of the trust-region on the optimal solution and (2) the effect of clustering on the computation time and the optimal objective value. Additionally, the use of simulated data provides us with a ground truth to use in evaluating the quality of the prescriptions.

4.1. Conceptual Model

The optimization model is a combination of a capacitated, multicommodity network flow model and a diet model with constraints for nutrition levels and food basket palatability.

The sets used to define the constraints and the objective function are displayed in Table 1. We have three different sets of nodes, and the set of commodities contains all the foods available for procurement during the food aid operation.

Figure 5. (Color online) Trust Region Enlarged Using a Hyperball with Radius ϵ Around Each Sample in $CH(X)$

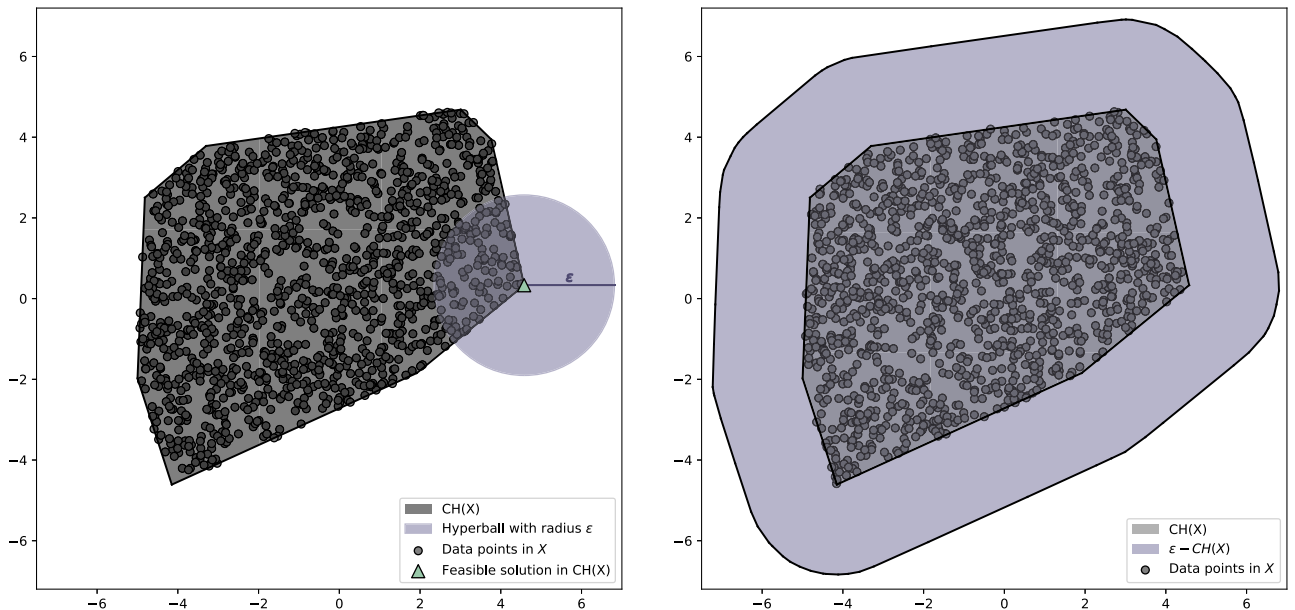


Table 1. Definition of the Sets Used in the WFP Model

Set	Description
$\mathcal{N}_S$	Set of source nodes
$\mathcal{N}_T$	Set of transshipment nodes
$\mathcal{N}_D$	Set of delivery nodes
$\mathcal{K}$	Set of commodities ($k \in \mathcal{K}$)
$\mathcal{L}$	Set of nutrients ($l \in \mathcal{L}$)

The parameters used in the model are displayed in Table 2. The costs used in the objective function concern transportation (p^T) and procurement (p^P). The amount of food to deliver depends on the demand (d) and the number of feeding days ($days$). The nutritional requirements ($nutreq$) and nutritional values ($nutval$) are detailed in Online Appendix C. The parameter γ is needed to convert the metric tons used in the supply chain constraints to the grams used in the nutritional constraints. The parameter t is used as a lower bound on the food basket palatability. The values of these parameters are based on those used by Peters et al. (2021).

The decision variables are shown in Table 3. The flow variables F_{ijk} are defined as the metric tons of a commodity k transported from node i to j . The variable x_k represents the average daily ration per beneficiary for commodity k . The variable y refers to the palatability of the food basket.

The full model formulation is as follows:

$$\min_{x, y, F} \sum_{i \in \mathcal{N}_S} \sum_{j \in \mathcal{N}_T \cup \mathcal{N}_D} \sum_{k \in \mathcal{K}} p_{ik}^P F_{ijk} + \sum_{i \in \mathcal{N}_S \cup \mathcal{N}_T} \sum_{j \in \mathcal{N}_T \cup \mathcal{N}_D} \sum_{k \in \mathcal{K}} p_{ijk}^T F_{ijk} \quad (6a)$$

$$\text{s.t. } \sum_{j \in \mathcal{N}_T} F_{ijk} = \sum_{j \in \mathcal{N}_T} F_{jik}, \quad i \in \mathcal{N}_T, \quad k \in \mathcal{K}, \quad (6b)$$

$$\sum_{j \in \mathcal{N}_S \cup \mathcal{N}_T} \gamma F_{ijk} = d_i x_k days, \quad i \in \mathcal{N}_D, \quad k \in \mathcal{K}, \quad (6c)$$

$$\sum_{k \in \mathcal{K}} Nutval_{kl} x_k \geq Nutreq_l, \quad l \in \mathcal{L}, \quad (6d)$$

$$x_{salt} = 5, \quad (6e)$$

$$x_{sugar} = 20, \quad (6f)$$

Table 2. Definition of the Parameters Used in the WFP Model

Parameter	Description
γ	Conversion rate from metric tons (mt) to grams (g)
d_i	Number of beneficiaries at delivery point $i \in \mathcal{N}_D$
$days$	Number of feeding days
$nutreq_l$	Nutritional requirement for nutrient $l \in \mathcal{L}$ (grams/person/day)
$nutval_{kl}$	Nutritional value for nutrient $l \in \mathcal{L}$ per gram of commodity $k \in \mathcal{K}$
p_{ik}^P	Procurement cost (in \$ / mt) of commodity k from source $i \in \mathcal{N}_S$
p_{ijk}^T	Transportation cost (in \$ / mt) of commodity k from node $i \in \mathcal{N}_S \cup \mathcal{N}_T$ to node $j \in \mathcal{N}_T \cup \mathcal{N}_D$
t	Palatability lower bound

Table 3. Definition of the Variables Used in the WFP Model

Variable	Description
F_{ijk}	Metric tons of commodity $k \in \mathcal{K}$ transported between node i and node j
x_k	Grams of commodity $k \in \mathcal{K}$ in the food basket
y	Food basket palatability

$$y \geq t, \quad (6g)$$

$$y = \hat{h}(x), \quad (6h)$$

$$F_{ijk}, x_k \geq 0, \quad i, j \in \mathcal{N}, \quad k \in \mathcal{K}. \quad (6i)$$

The objective function consists of two components, procurement costs and transportation costs. Constraints (6b) are used to balance the network flow, namely to ensure that the inflow and the outflow of a commodity are equal for each transshipment node. Constraints (6c) state that flow into a delivery node has to be equal to its demand, which is defined by the number of beneficiaries times the daily ration for commodity k times the feeding days. Constraints (6d) guarantee an optimal solution that meets the nutritional requirements. Constraints (6e) and (6f) force the amount of salt and sugar to be 5 and 20 grams, respectively. Constraint (6g) requires the food basket palatability (y), defined by means of a predictive model (6h), to be greater than a threshold (t). Last, nonnegativity Constraints (6i) are added for all commodity flows and commodity rations.

4.2. Data Set and Predictive Models

To evaluate the ability of our framework to learn and implement the palatability constraints, we use a simulator to generate diets with varying palatabilities. Each sample is defined by 25 features representing the amount (in grams) of all commodities that make up the food basket. We then use a ground truth function to assign each food basket a palatability between zero and one, where one corresponds to a perfectly palatable basket and zero to an inedible basket. This function is based on suggestions provided by WFP experts and complete details are outlined in Online Appendix C.1. The data are then balanced to ensure that a wide variety of palatability scores are represented in the data set. The final data used to learn the palatability constraint consists of 121,589 samples. Two examples of daily food baskets and their respective palatability scores are shown in Table 4. In this case study, we use a palatability lower bound (t) of 0.5 for our learned constraint.

The next step of the framework involves training and choosing the predictive model that best approximates the unknown constraint. The predictive models used to learn the palatability constraints are those discussed in Section 2, namely LR, SVM, CART, RF, GBM with decision trees as base-learners, and MLP with ReLU activation function.

Table 4. Two Examples of Daily Food Baskets

Commodity	Basket 1 amount (g)	Basket 2 amount (g)
DSM	31.9	33.9
Chickpeas	—	75.7
Lentils	41	—
Maize meal	48.9	—
Meat	—	17.2
Oil	22	28.6
Salt	5	5
Sugar	20	20
Wheat	384.2	131.2
Wheat flour	—	261.3
WSB	67.3	59.8
Palatability score	0.436	0.741

Note. DSM, dried skim milk; WSB, wheat soya blend.

4.3. Optimization Results

The experiments are executed using OptiCL jointly with Gurobi v9.1 (Gurobi Optimization LLC 2021) as the optimization solver. Table 5 reports the performances of the predictive models evaluated both for the validation set and for the prescriptions after being embedded into the optimization model. The table also compares the performance of the optimization with and without the trust region. The column Validation MSE gives the mean squared error (MSE) of each model obtained in cross-validation during model selection. Although all scores in this column are desirably low, the MLP model significantly achieves the lowest error during this validation phase. The column MSE gives the MSE of the predictive models once embedded into the optimization problem to evaluate how well the predictions for the optimal solutions match their true palatabilities (computed using the simulator). It is found using 100 optimal solutions of the optimization model generated with different cost vectors. The MLP model exhibits the best performance (0.055) in this context, showing its ability to model the palatability constraint better than all other methods.

Table 5. Predictive Models Performances for the Validation Set (Validation MSE) and Prescriptions After Being Embedded into the Optimization Model with (MSE-TR) and Without the Trust Region (MSE)

Model	Validation			Time (SD): seconds	Time-TR (SD): seconds
	MSE	MSE	MSE-TR		
LR	0.046	0.256	0.042	0.003 (0.0008)	1.813 (0.204)
SVM	0.019	0.226	0.027	0.003 (0.0006)	1.786 (0.208)
CART	0.014	0.273	0.059	0.012 (0.0030)	7.495 (5.869)
RF	0.018	0.252	0.025	0.248 (0.1050)	30.128 (13.917)
GBM	0.006	0.250	0.017	0.513 (0.4562)	60.032 (41.685)
MLP	0.001	0.055	0.001	14.905 (41.764)	28.405 (23.339)

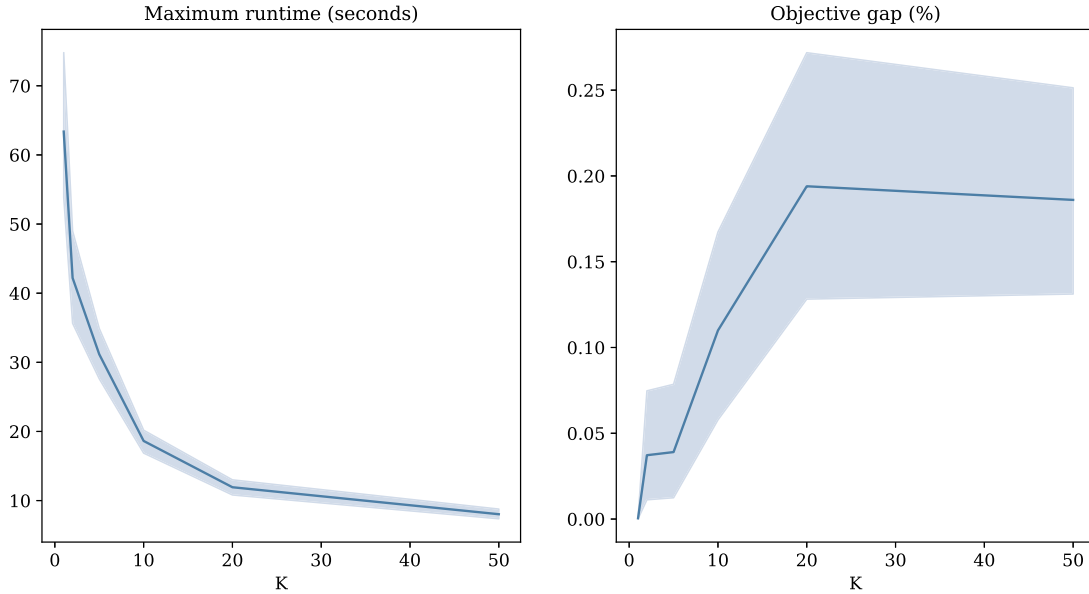
Notes. The last two columns show the average computation time in seconds and its standard deviation (SD) required to solve the optimization model with (Time-TR) and without the trust region (Time). Runtimes reported using an Intel i7-8665U 1.9-GHz CPU, with 16 GB of RAM (Windows 10 environment).

4.3.1. Benefit of Trust Region. Table 5 shows that when the trust region is used (MSE-TR), the MSEs obtained by all models are now much closer to the results from the validation phase. This shows the benefit of using the trust region as discussed in Section 2.3 to prevent extrapolation. With the trust region included, the MLP model also exhibits the lowest MSE (0.001). The improved performance seen with the inclusion of the trust region does come at the expense of computation speed. The column Time-TR shows the average computation time in seconds and its standard deviation (SD) with trust region constraints included. In all cases, the computation time has clearly increased when compared against the computation time required without the trust region (column Time). This is, however, acceptable, as significantly more accurate results are obtained with the trust region.

4.3.2. Benefit of Clustering. The large data set used in this case study makes the use of the trust region expensive in terms of time required to solve the final optimization model. Although the column selection algorithm described in Section 2.3 is ideal for significantly reducing the computation time, optimization models that require binary variables, either for embedding an ML model or to represent decision variables, would require column selection to be combined with a branch-and-bound algorithm. However, in this more general MIO case, it is possible to divide the data set into clusters and solve in parallel an MIO for each cluster. By using parallelization, the total solution time can be expected to be equal to the longest time required to solve any single cluster's MIO. Contrary to column selection, the use of clusters can result in more conservative solutions; the trust region gets smaller with more clusters and prevents the model from finding solutions that are convex combinations of members of different clusters. However, as described in Section 2.3, solutions that lie between clusters may in fact reside in low-density areas of the feature space that should not be included in the trust region. In this sense, the loss in the objective value might actually coincide with more trustable solutions.

Figure 6 shows the effect of clusters in solving Model (6a)–(6i) with GBM as the predictive model used to learn the palatability constraint. K-means is used to partition the data set into K clusters, and the reported values are averaged over 100 iterations. In the left graph, we report the maximum runtime distribution across clusters needed to solve the different MIOs in parallel. In the right graph, we have the distributions of optimality gap, that is, the relative difference between the optimal solution obtained with clusters compared with the solution obtained with no clustering. In this case study, the use of clusters significantly decreases the runtime (89.2% speed up with $K = 50$) while still obtaining near-optimal solutions (less than 0.25% average gap with $K = 50$). We

Figure 6. (Color online) Effect of the Number of Clusters (K) on the Computation Time and Optimality Gap Across Clusters, with Bootstrapped 95% Confidence Intervals



observe that the trends are not necessarily monotonic in K . It is possible that a certain choice of K may lead to a suboptimal solution, whereas a larger value of K may preserve the optimal solution as the convex combination of points within a single cluster.

4.4. Robustness Results

In these experiments, we assess the performance of the nominal and robust models. We consider three dimensions of performance: (1) true constraint satisfaction, (2) objective function value, and (3) runtime. The synthetic data used in this case study allows us to evaluate true palatability and constraint satisfaction as these parameters vary. This is the primary goal of the model wrapper ensemble approach, to improve feasibility and make solutions that are robust to any single learned estimator.

We hypothesize that as our models become more conservative, we will more reliably satisfy the desired palatability constraint with some toll on the objective function. Additionally, embedding multiple models or characterizing uncertainty sets introduces computational complexity over a single nominal model. In this section, we compare the tradeoffs in these metrics as we consider different notions of robustness and vary our conservativeness. We are able to evaluate whether the true palatability meets the constraint threshold because palatability is defined through a known function. As with the previous experiments, we solve the palatability problem with 100 different realizations of the cost vector and average the results.

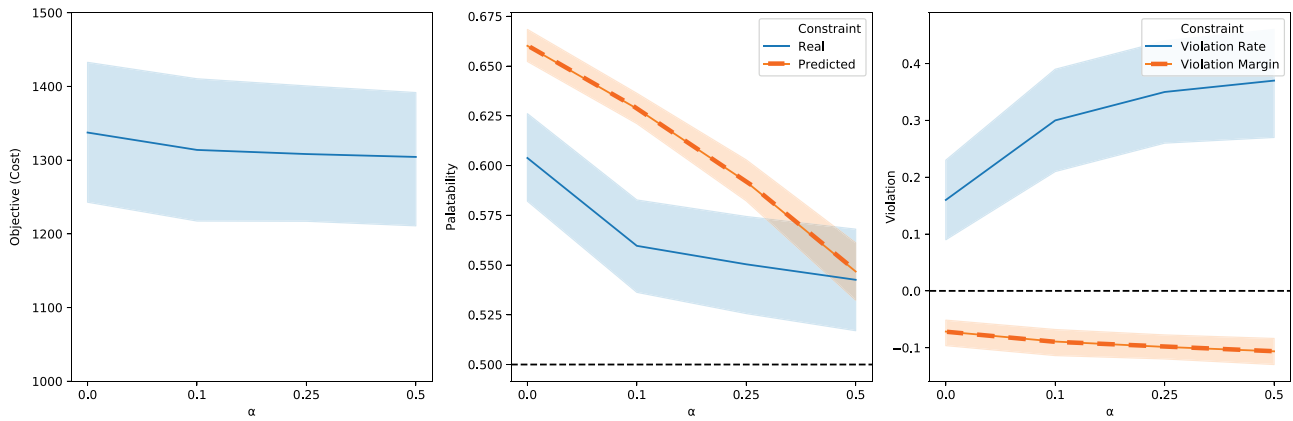
The results explore the effect of the α (violation limit) on cost and palatability in the WFP case study. Additional results on runtime and experiments with varied

estimators (P) are included in Online Appendix C.3. As the results demonstrate, the robustness parameters yield solutions that vary in their conservativeness and runtime. There is not a single set of optimal parameters. Rather, it is highly dependent on the use case, including factors like the stakes of the decision and the allowable turnaround time to generate solutions.

4.4.1. Multiple Embedded Models. We first consider the impact of the model wrapper approach in the WFP problem. We compare different ways of embedding the palatability constraint, both using multiple estimators of a single model class and an ensemble containing multiple model classes. We run the experiments on a random sample of 1,000 observations in the original WFP data set. Within a single model class, we vary the number of estimators ($P \in [2, 5, 10, 25]$) and the violation limit ($\alpha \in [0, 0.1, 0.2, 0.5]$ or applying a mean constraint). Each estimator is obtained using a bootstrap sample (proportion = 0.5) of the underlying data. We compute metrics (one to three) for each variant to compare the tradeoffs in palatability (constraint satisfaction) and cost (objective function value).

Figure 7 presents the results for a decision tree with $p = 25$ and palatability threshold (τ) equal to 0.5. The left figure shows the trade off between palatability and the objective as the violation limit (α) varies. As expected, improvements in palatability (when α decreases) lead to increases in the total cost. However, we observe that a violation limit of 0.0 (versus 0.5) leads to an 11.3% improvement in real palatability (20.8% improvement in predicted palatability), with a relatively modest 2.5%

Figure 7. (Color online) Comparison of CART Models on Objective Function and Constraint Satisfaction



increase in cost. The center and right figure show how palatability and violations vary with α . Palatability increases and violations decrease with lower α . Both the violation rate (proportion of iterations with real palatability < 0.5) and violation margin (average distance to palatability threshold in cases where there is a violation) decrease with lower α . This experiment demonstrates how the α parameter effectively controls the model's robustness as measured by constraint satisfaction. The approach has the advantage of parameterizing the violation limit, allowing us to explicitly control the model's conservativeness and evaluate constraint-objective tradeoffs.

Online Appendix C.3 reports further results for other model classes and runtime experiments.

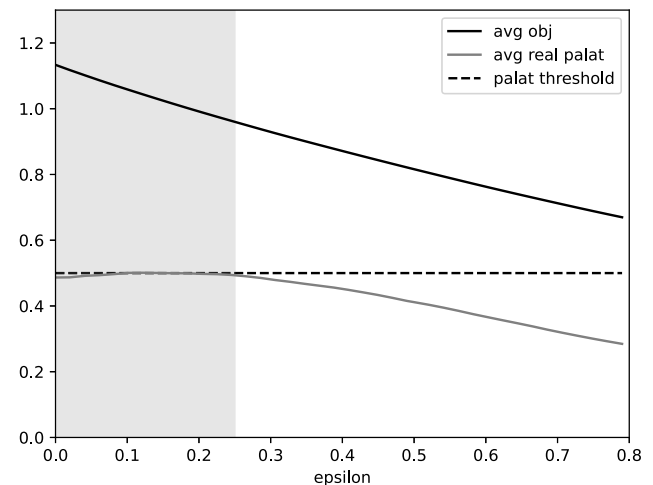
4.4.2. Enlarged Trust Region. To evaluate the effects of the enlarged trust region on the optimal solution, we use a simplified version of Problem (6a)–(6i) where the only constraints are on the predictive model embedding, the palatability lower bound, and the ϵ -CH. In Figure 8, we show how the objective function value and true palatability score vary according to different values of $\epsilon \in [0, 0.8]$. The results are obtained by averaging more than 200 iterations with randomly generated cost vectors and using a decision tree as a predictive model to represent the palatability outcome. As expected, the objective value improves as ϵ increases. More interesting is the true palatability score that stays around the imposed lower bound of 0.5 for values of ϵ smaller than 0.25. This means that the predictive model is able to generalize even outside the CH as long as the optimal solution is not too far from it.

5. Case Study: Chemotherapy Regimen Design

In this case study, we extend the work of Bertsimas et al. (2016) in the design of chemotherapy regimens for advanced gastric cancer. Late-stage gastric cancer has a poor prognosis with limited treatment options (Yang et al.

2011). This has motivated significant research interest and clinical trials (National Cancer Institute 2021). In Bertsimas et al. (2016), the authors pose the question of algorithmically identifying promising chemotherapy regimens for new clinical trials based on existing trial results. They construct a database of clinical trial treatment arms that includes cohort and study characteristics, the prescribed chemotherapy regimen, and various outcomes. Given a new study cohort and study characteristics, they optimize a chemotherapy regimen to maximize the cohort's survival subject to a constraint on overall toxicity. The original work uses linear regression models to predict survival and toxicity, and it constrains a single toxicity measure. In this work, we leverage a richer class of ML methods and more granular outcome measures. This offers benefits through higher performing predictive models and more clinically relevant constraints.

Figure 8. Effect of ϵ -CH on the Objective Value and Predictive Model Performance with Respect to the Optimal Solution



Note. The values are obtained as an average of 200 iterations.

Chemotherapy regimens are particularly challenging to optimize because they involve multiple drugs given at potentially varying dosages, and they present risks for multiple adverse events that must be managed. This example highlights the generalizability of our framework to complex domains with multiple decisions and learned functions. The treatment variables in this problem consist of both binary and continuous elements, which are easily incorporated through our use of MIO. We have several learned constraints which must be simultaneously satisfied, and we also learn the objective function directly as a predictive model.

5.1. Conceptual Model

The use of clinical trial data forces us to consider each cohort as an observation rather than an individual because only aggregate measures are available. Thus, our model optimizes a cohort's treatment. The contextual variables (w) consist of various cohort and study summary variables. The inclusion of fixed, that is, non-optimization, features allows us to account for differences in baseline health status and risk across study cohorts. These features are included in the predictive models but then are fixed in the optimization model to reflect the group for whom we are generating a prescription. We assume that there are no unobserved confounding variables in this prescriptive setting.

The treatment variables (x) encode a chemotherapy regimen. A regimen is defined by a set of drugs, each with an administration schedule of potentially varied dosages throughout a chemotherapy cycle. We characterize a regimen by drug indicators and each drug's average daily dose and maximum instantaneous dose in the cycle:

$$\begin{aligned} x_b^d &= \mathbb{I}(\text{drug } d \text{ is administered}), \\ x_a^d &= \text{average daily dose of drug } d, \\ x_i^d &= \text{maximum instantaneous dose of drug } d. \end{aligned}$$

This allows us to differentiate between low-intensity, high-frequency and high-intensity, low-frequency dosing strategies. The outcomes of interest (y) consist of overall survival, to be included as the objective (y_{OS}), and various toxicities, to be included as constraints ($y_i, i \in \mathcal{Y}_C$).

To determine the optimal chemotherapy regimen x for a new study cohort with characteristics w , we formulate the following MIO:

$$\begin{aligned} \min_{x, y} \quad & y_{OS} \\ \text{s.t.} \quad & y_i \leq \tau_i, \quad i \in \mathcal{Y}_C, \\ & y_i = \hat{h}_i(x, w), \quad i \in \mathcal{Y}_C, \\ & y_{OS} = \hat{h}_{OS}(x, w), \\ & \sum_d x_b^d \leq 3, \\ & x_b \in \{0, 1\}^d, \\ & x \in \mathcal{X}(w). \end{aligned}$$

In this case study, we learn the full objective. However, this model could easily incorporate deterministic components to optimize as additional weighted terms in the objective. We include one domain-driven constraint, enforcing a maximum regimen combination of three drugs.

The trust region, $\mathcal{X}(w)$, plays two crucial roles in the formulation. First, it ensures that the predictive models are applied within their valid bounds and not inappropriately extrapolated. It also naturally enforces a notion of “clinically reasonable” treatments. It prevents drugs from being prescribed at doses outside of previously observed bounds, and it requires that the drug combination must have been previously seen (although potentially in different doses). It is nontrivial to explicitly characterize what constitutes a realistic treatment, and the convex hull provides a data-driven solution that integrates directly into the model framework. Furthermore, the convex hull implicitly enforces logical constraints between the different dimensions of x . For example, a drug's average and instantaneous dose must be zero if the drug's binary indicator is set to zero: This does not need to be explicitly included as a constraint because this is true for all observed treatment regimens. The only explicit constraint required here is that the indicator variables x_b are binary.

5.2. Data Set

Our data consist of 495 clinical trial arms from 1979 to 2012 (Bertsimas et al. 2016). We consider nine contextual variables, including the average patient age and breakdown of primary cancer site. There are 28 unique drugs that appear in multiple arms of the training set, yielding 84 decision variables. We include several dose-limiting toxicities (DLTs) for our constraint set: Grade 3/4 constitutional toxicity, gastrointestinal toxicity, and infection, as well as Grade 4 blood toxicity. As the name suggests, these are chemotherapy side effects that are severe enough to affect the course of treatment. We also consider incidence of any dose-limiting toxicity (Any DLT), which aggregates over a superset of these DLTs.

We apply a temporal split, training the predictive models on trial arms through 2008 and generating prescriptions for the trial arms in 2009–2012. The final training set consists of 320 observations, and the final testing set consists of 96 observations. The full feature set, inclusion criteria, and data processing details are included in Online Appendix D.1.

To define the trust region, we take the convex hull of the treatment variables (x) on the training set. This aligns with the temporal split setting, in which we are generating prescriptions going forward based on an existing set of past treatment decisions. In general, it is preferable to define the convex hull with respect to both x and w as discussed in Online Appendix B.1, but this

does not apply well with a temporal split. Our data include the study year as a feature to incorporate temporal effects, and so our test set observations will definitionally fall outside of the convex hull defined by the observed (x, w) in our training set.

5.3. Predictive Models

Several ML models are trained for each outcome of interest using cross-validation for parameter tuning, and the best model is selected based on the validation criterion. We use function learning for all toxicities, directly predicting the toxicity incidence and applying an upper bound threshold within the optimization model.

Based on the model selection procedure, overall DLT, gastrointestinal toxicity, and overall survival are predicted using GBM models. Blood toxicity and infection are predicted using linear models, and constitutional toxicity is predicted with a RF model. This demonstrates the advantage of learning with multiple model classes; no single method dominates in predictive performance. A complete comparison of the considered models is included in Online Appendix D.2.

5.4. Evaluation Framework

We generate prescriptions using the optimization model outlined in Section 5.1, with the embedded model choices specified in Section 5.3. To evaluate the quality of our prescriptions, we must estimate the outcomes under various treatment alternatives. This evaluation task is notoriously challenging due to the lack of counterfactuals. In particular, we only know the true outcomes for observed cohort-treatment pairs and do not have information on potential unobserved combinations. We propose an evaluation scheme that leverages a “ground truth” ensemble (GT ensemble). We train several ML models using all data from the study. These models are not embedded in an MIO model, so we are able to consider a broader set of methods in the ensemble. We then predict each outcome by averaging across all models in the ensemble. This approach allows us to capture the maximal knowledge scenario. Furthermore, such a “consensus” approach of

combining ML models has been shown to improve predictive performance and is more robust to individual model error (Bertsimas et al. 2021). The full details of the ensemble models and their predictive performances are included in Online Appendix D.3.

5.5. Optimization Results

We evaluate our model in multiple ways. We first consider the performance of our prescriptions against observed (given) treatments. We then explore the impact of learning multiple subconstraints rather than a single aggregate toxicity constraint. All optimization models have the following shared parameters: toxicity upper bound of 0.6 quantile (as observed in training data) and maximum violation of 25% for RF models. We report results for all test set observations with a feasible solution. It is possible that an observation has no feasible solution, implying that there is not a suitable drug combination lying within the convex hull for this cohort based on the toxicity requirements. These cases could be further investigated through a sensitivity analysis by relaxing the toxicity constraints or enlarging the trust region. With clinical guidance, one could evaluate the modifications required to make the solution feasible and the clinical appropriateness of such relaxations.

Table 6 reports the predicted outcomes under two constraint approaches: (1) constraining each toxicity separately (All constraints) and (2) constraining a single aggregate toxicity measure (DLT only). For each cohort in the test set, we generate predictions for all outcomes of interest under both prescription schemes and compute the relative change of our prescribed outcome from the given outcome predictions.

5.5.1. Benefit of Prescriptive Scheme. We begin by evaluating our proposed prescriptive scheme (All constraints) against the observed actual treatments. For example, under the GT ensemble scheme, 84.7% of cohorts satisfied the overall DLT constraint under the given treatment compared with 94.1% under the proposed treatment. This yields an improvement of 11.10%. We obtain a significant improvement in survival (11.40%)

Table 6. Comparison of Outcomes Under Given Treatment Regimen, Regimen Prescribed When Only Constraining the Aggregate Toxicity, and Regimen Prescribed Under Our Full Model

	All constraints			DLT only	
	Given (SD)	Prescribed (SD)	Percent change	Prescribed (SD)	Percent change
Any DLT	0.847 (0.362)	0.941 (0.237)	11.10%	0.906 (0.294)	6.90%
Blood	0.812 (0.393)	0.824 (0.383)	1.40%	0.706 (0.458)	−13.00%
Constitutional	0.953 (0.213)	1.000 (0.000)	4.90%	1.000 (0.000)	4.90%
Infection	0.882 (0.324)	0.894 (0.310)	1.30%	0.800 (0.402)	−9.30%
Gastrointestinal	0.800 (0.402)	1.000 (0.000)	25.00%	1.000 (0.000)	25.00%
Overall survival	10.855 (1.939)	12.092 (1.470)	11.40%	12.468 (1.430)	14.90%

Notes. We report the mean and standard deviation (SD) of constraint satisfaction (binary indicator) and overall survival (months) across the test set. The relative change is reported against the given treatment.

while also improving toxicity limit satisfaction across all individual toxicities. Using the GT ensemble, we see toxicity satisfaction improvements between 1.3% and 25.0%. We note that because toxicity violations are reported using the average incidence for each cohort, and the constraint limits are toxicity specific, it is possible for a single DLT's incidence to be over the allowable limit, whereas the overall "Any DLT" rate is not.

5.5.2. Benefit of Multiple Constraints. Table 6 also illustrates the value of enforcing constraints on each individual toxicity rather than as a single measure. When only constraining the aggregate toxicity measure (DLT Only), the resultant prescriptions actually have lower constraint satisfaction for blood toxicity and infection than the baseline given regimens. By constraining multiple measures, we are able to improve across all individual toxicities. The fully constrained model actually improves the overall DLT measure satisfaction, suggesting that the inclusion of these "subconstraints" also makes the aggregate constraint more robust. This improvement does come at the expense of slightly lower survival between the All and DLT only models (-0.38 months), but we note that incurring the individual toxicities that are violated in the DLT only model would likely make the treatment unviable.

6. Discussion

Our experimental results illustrate the benefits of our constraint learning framework in data-driven decision making in two problem settings: food basket recommendations for the WFP and chemotherapy regimens for advanced gastric cancer. The quantitative results show an improvement in predictive performance when incorporating the trust region and learning from multiple candidate model classes. Our framework scales to large problem sizes, enabled by efficient formulations and tailored approaches to specific problem structures. Our approach for efficiently learning the trust region also has broad applicability in one-class constraint learning.

The nominal problem formulation is strengthened by embedding multiple models for a single constraint rather than relying on a single learned function. This notion of robustness is particularly important in the context of learning constraints: whereas mis-specifications in learned objective functions can lead to suboptimal outcomes, a mis-specified constraint can lead to infeasible solutions. Finally, our software exposes the model ensemble construction and trust region enlargement options directly through user-specified parameters. This allows an end user to directly evaluate tradeoffs in objective value and constraint satisfaction, as the problem's real-world context often shapes the level of desired conservatism.

We recognize several opportunities to further extend this framework. Our work naturally relates to the causal inference literature and individual treatment effect estimation (Athey and Imbens 2016, Shalit et al. 2017). These methods do not directly translate to our problem setting; existing work generally assumes highly structured treatment alternatives (e.g., binary treatment versus control) or a single continuous treatment (e.g., dosing), whereas we allow more general decision structures. In future work, we are interested in incorporating ideas from causal inference to relax the assumption of unobserved confounders.

Additionally, our framework is dependent on the quality of the underlying predictive models. We constrain and optimize point predictions from our embedded models. This can be problematic in the case of model misspecification, a known shortcoming of "predict-then-optimize" methods (Elmachtoub and Grigas 2021). We mitigate this concern in two ways. First, our model selection procedure allows us to obtain higher-quality predictive models by capturing several possible functional relationships. Second, our model wrapper approach for embedding a single constraint with an ensemble of models allows us to directly control our robustness to the predictions of individual learners. In future work, there is an opportunity to incorporate ideas from robust optimization to directly account for prediction uncertainty in individual model classes. Although this has been addressed in the linear case (Goldfarb and Iyengar 2003), it remains an open area of research in more general ML methods.

In this work, we present a unified framework for optimization with learned constraints that leverages both ML and MIO for data-driven decision making. Our work flexibly learns problem constraints and objectives with supervised learning and incorporates them into a larger optimization problem of interest. We also learn the trust region, providing more credible recommendations and improving predictive performance, and accomplish this efficiently using column generation and unsupervised learning. The generality of our method allows us to tackle quite complex decision settings, such as chemotherapy optimization but also includes tailored approaches for more efficiently solving specific problem types. Finally, we implement this as a Python software package (OptiCL) to enable practitioner use. We envision that OptiCL's methodology will be added to state-of-the-art optimization modeling software packages.

Acknowledgments

The authors thank the anonymous reviewers and editorial team for valuable feedback on this work. Donato Maragno and Holly Wiberg contributed equally to this work. Any opinion, findings, and conclusions or recommendations expressed in this material are those of the authors and do not necessarily reflect the views of the National Science Foundation.

References

- Amos B, Xu L, Kolter JZ (2017) Input convex neural networks. *Int. Conf. Mach. Learn.* (PMLR, New York), 146–155.
- Anderson R, Huchette J, Ma W, Tjandraatmadja C, Vielma JP (2020) Strong mixed-integer programming formulations for trained neural networks. *Math. Programming* 183(1–2):3–39.
- Athey S, Imbens G (2016) Recursive partitioning for heterogeneous causal effects. *Proc. National Acad. Sci. USA* 113(27):7353–7360.
- Balestriero R, Pesenti J, LeCun Y (2021) Learning in high dimension always amounts to extrapolation. Preprint, submitted October 18, <https://arxiv.org/abs/2110.09485>.
- Bengio Y, Lodi A, Prouvost A (2021) Machine learning for combinatorial optimization: A methodological tour d’horizon. *Eur. J. Oper. Res.* 290(2):405–421.
- Bergman D, Huang T, Brooks P, Lodi A, Raghunathan AU (2022) JANOS: An integrated predictive and prescriptive modeling framework. *INFORMS J. Comput.* 34(2):807–816.
- Bertsimas D, Dunn J (2017) Optimal classification trees. *Machine Learn.* 106(7):1039–1082.
- Bertsimas D, Dunn J (2018) *Machine Learning Under a Modern Optimization Lens* (Dynamic Ideas, Belmont, MA).
- Bertsimas D, Kallus N (2020) From predictive to prescriptive analytics. *Management Sci.* 66(3):1025–1044.
- Bertsimas D, O’Hair A, Relyea S, Silberholz J (2016) An analytics approach to designing combination chemotherapy regimens for cancer. *Management Sci.* 62(5):1511–1531.
- Bertsimas D, Borenstein A, Mingardi L, Nohadani O, Orfanoudaki A, Stellato B, Wiberg H, et al. (2021) Personalized prescription of ACEI/ARBs for hypertensive COVID-19 patients. *Healthcare Management Sci.* 24(2):339–355.
- Biggs M, Hariss R, Perakis G (2021) Optimizing objective functions determined from random forests. Preprint, submitted June 16, <https://dx.doi.org/10.2139/ssrn.2986630>.
- Bonfietti A, Lombardi M, Milano M (2015) Embedding decision trees and random forests in constraint programming. Michel L, ed. *Integration of AI and OR Techniques in Constraint Programming. CPAIOR 2015. Lecture Notes in Computer Science*, vol. 9075 (Springer, Cham, Switzerland), 74–90.
- Breiman L (2001) Random forests. *Machine Learn.* 45(1):5–32.
- Breiman L, Friedman JH, Olshen RA, Stone CJ (1984) *Classification and Regression Trees* (Routledge, London).
- Chen Y, Shi Y, Zhang B (2020) Input convex neural networks for optimal voltage regulation. Preprint, submitted February 20, <https://arxiv.org/abs/2002.08684>.
- Cremer JL, Konstantelos I, Tindemans SH, Strbac G (2019) Data-driven power system operation: Exploring the balance between cost and risk. *IEEE Trans. Power Systems* 34(1):791–801.
- Ebert T, Belz J, Nelles O (2014) Interpolation and extrapolation: Comparison of definitions and survey of algorithms for convex and concave hulls. *Proc. IEEE Sympos. on Computational Intelligence and Data Mining* (IEEE, Piscataway, NJ), 310–314.
- Elmachtoub AN, Grigas P (2021) Smart “predict, then optimize”. *Management Sci.* 68(1):9–26.
- Fajemisin AO, Maragno D, den Hertog D (2023) Optimization with constraint learning: A framework and survey. *Eur. J. Oper. Res.* ePub ahead of print May 3, <https://doi.org/10.1016/j.ejor.2023.04.041>.
- Goldfarb D, Iyengar G (2003) Robust portfolio selection problems. *Math. Oper. Res.* 28(1):1–38.
- Goodfellow IJ, Shlens J, Szegedy C (2015) Explaining and harnessing adversarial examples. Preprint, submitted December 20, <https://arxiv.org/abs/1412.6572>.
- Grimstad B, Andersson H (2019) ReLU networks as surrogate models in mixed-integer linear programs. *Computers Chemical Engrg.* 131:106580.
- Gurobi Optimization LLC (2021) Gurobi optimizer reference manual. Accessed June 28, 2021, <https://www.gurobi.com>.
- Gutierrez-Martinez VJ, Cañizares CA, Fuerte-Esquivel CR, Pizano-Martinez A, Gu X (2011) Neural-network security-boundary constrained optimal power flow. *IEEE Trans. Power Systems* 26(1): 63–72.
- Halilbasic L, Thams F, Venzke A, Chatzivasileiadis S, Pinson P (2018) Data-driven security-constrained AC-OPF for operations and markets. *Proc. 20th Power Systems Computation Conf.* (IEEE, Piscataway, NJ), 1–7.
- Kleijnen JP (2018) Design and analysis of simulation experiments (Springer, Berlin).
- Kudła P, Pawlak TP (2018) One-class synthesis of constraints for Mixed-Integer Linear Programming with C4.5 decision trees. *Applied Soft Comput.* 68:1–12.
- Lombardi M, Milano M, Bartolini A (2017) Empirical decision model learning. *Artificial Intelligence* 244:343–367.
- Mišić VV (2020) Optimization of tree ensembles. *Oper. Res.* 68(5): 1605–1624.
- Murzakhanov I, Venzke A, Misyris GS, Chatzivasileiadis S (2022) Neural networks for encoding dynamic security-constrained optimal power flow. Preprint, submitted March 17, <https://arxiv.org/abs/2003.07939>.
- National Cancer Institute (2021) Treatment clinical trials for gastric (stomach) cancer. Accessed October 26, 2021, <https://www.cancer.gov/about-cancer/treatment/clinical-trials/disease/stomach-cancer/treatment>.
- Pawlak TP (2019) Synthesis of mathematical programming models with one-class evolutionary strategies. *Swarm Evolutionary Comput.* 44:335–348.
- Pawlak TP, Krawiec K (2019) Synthesis of constraints for mathematical programming with one-class genetic programming. *IEEE Trans. Evolutionary Comput.* 23(1):117–129.
- Pawlak TP, Litwiniuk B (2021) Ellipsoidal one-class constraint acquisition for quadratically constrained programming. *Eur. J. Oper. Res.* 293(1):36–49.
- Peters K, Silva S, Gonçalves R, Kavelj M, Fleuren H, den Hertog D, Ergun O, et al. (2021) The nutritious supply chain: Optimizing humanitarian food assistance. *INFORMS J. Optim.* 3(2):200–226.
- Schweidtmann AM, Mitsos A (2019) Deterministic global optimization with artificial neural networks embedded. *J. Optim. Theory Appl.* 180(3):925–948.
- Shalit U, Johansson FD, Sontag D (2017) Estimating individual treatment effect: Generalization bounds and algorithms. *Proc. Internat. Conf. on Machine Learn.* (PMLR, New York), 3076–3085.
- Skiena SS (2008) *The Algorithm Design Manual*, 2nd ed. (Springer Publishing Company, Berlin).
- Spyros C (2020) From decision trees and neural networks to MILP: Power system optimization considering dynamic stability constraints. *Proc. Eur. Control Conf.* (IEEE, Piscataway, NJ), 594–594.
- Sroka D, Pawlak TP (2018) One-class constraint acquisition with local search. *Proc. Genetic and Evolutionary Comput. Conf.* (Association for Computing Machinery, New York), 363–370.
- Thams F, Halilbaşı L, Pinson P, Chatzivasileiadis S, Eriksson R (2017) Data-driven security-constrained OPF. *Proc. 10th Bulk Power Systems Dynamic Control Sympos.* (IEEE, Piscataway, NJ), 1–10.
- Verwer S, Zhang Y, Ye QC (2017) Auction optimization using regression trees and linear models as integer programs. *Artificial Intelligence* 244:368–395.
- Yang D, Hendifar A, Lenz C, Togawa K, Lenz F, Lurje G, Pohl A, et al. (2011) Survival of metastatic gastric cancer: Significance of age, sex and race/ethnicity. *J. Gastrointestinal Oncology* 2(2):77–84.

Donato Maragno is a PhD student at the Department of Business Analytics, University of Amsterdam (Netherlands). His research interests focus on the intersection of optimization and machine learning with applications in explainable artificial intelligence and radiation therapy.

Holly Wiberg is an assistant professor of operations research and public policy at the Heinz College of Information Systems and public policy at Carnegie Mellon University. Her research focuses on the intersection of optimization and machine learning, particularly on the development and application of these areas for data-driven healthcare delivery.

Dimitris Bertsimas is the Boeing professor of operations research and associate dean of business analytics at Sloan School of Management, Massachusetts Institute of Technology. His research interests include machine learning and optimization and their applications in healthcare. He is currently editor in chief of *INFORMS Journal on Optimization*.

Ş. İlker Birbil is a professor of AI and optimization techniques for business and society at the University of Amsterdam. His research interests center around optimization methods in data

science and decision making. Lately, he is working on explainable artificial intelligence, optimization for machine learning, and data privacy in operations research.

Dick den Hertog is a professor of operations research at University of Amsterdam. His research interests cover various fields in linear and nonlinear optimization. His main focus has been on robust optimization, and he is also active in applying the theory in real-life applications. In particular, he is interested in applications that contribute to a better society. He is science-to-impact director of analytics for a Better World Institute, which he co-founded in 2022.

Adejuyigbe O. Fajemisin is a postdoctoral researcher at Amsterdam Business School (Netherlands). His research interests include artificial intelligence, operations research, and data-driven optimization. His goal is to collaborate with companies and nonprofit organizations to solve practical optimization problems and make the world a better place.